📋 Plano de Continuidade - Sistema de Inventário Protheus

🎉 Status Atual (CONCLUÍDO)

✅ Infraestrutura

- **Docker Compose** funcionando (`postgres`, `backend`, `redis`, `pgadmin`)
- **PostgreSQL** com banco inicializado e dados de teste
- **FastAPI** backend rodando na porta 8000
- **Documentação automática** em `/docs`

✅ Backend Python

- **Autenticação JWT** funcionando 100%
- **Login admin/admin123** testado e aprovado
- **Modelos SQLAlchemy** criados (User, Store, Product, etc.)
- **Conexão com PostgreSQL** funcionando
- **Sistema de segurança** robusto implementado

✅ Arquivos Criados

```
capul_inventario/
├── docker-compose.yml        ✅
├── .env                      ✅
├── backend/
│   ├── requirements.txt      ✅
│   ├── Dockerfile            ✅
│   ├── app/
│   │   ├── main.py           ✅
│   │   ├── core/
│   │   │   ├── config.py     ✅
│   │   │   ├── database.py   ✅
│   │   │   └── security.py   ✅
│   │   ├── models/
│   │   │   └── models.py     ✅
│   │   └── api/
│   │       └── auth.py       ✅
└── database/
    └── init.sql              ✅
```

✅ Endpoints Funcionando

- `GET /health` - Health check

- `GET /docs` - Documentação automática
- `GET /test/database` - Teste de conexão PostgreSQL
- `POST /api/v1/auth/login` - Login JWT (admin/admin123)
- `GET /api/v1/auth/test-user` - Buscar usuário admin

🚀 PRÓXIMOS PASSOS - PRIORIDADE 1

1. API de Produtos (30-45 min)

Objetivo: CRUD completo de produtos com autenticação JWT

Implementar:

- `GET /api/v1/products` - Listar produtos (com paginação)
- `POST /api/v1/products` - Criar produto
- `GET /api/v1/products/{id}` - Obter produto específico
- `PUT /api/v1/products/{id}` - Atualizar produto
- `DELETE /api/v1/products/{id}` - Excluir produto (soft delete)
- `GET /api/v1/products/search?q=termo` - Busca rápida
- `GET /api/v1/products/barcode/{code}` - Buscar por código de barras

Arquivos a criar:

- `backend/app/api/v1/endpoints/products.py`
- `backend/app/schemas/product_schemas.py`
- Atualizar `main.py` para incluir as rotas

Funcionalidades:

- Autenticação JWT obrigatória
- Filtro automático por loja do usuário logado
- Validação de dados com Pydantic
- Controle de permissões por role (Admin/Supervisor/Operator)

2. Dados de Teste (15 min)

Objetivo: Popular banco com produtos de exemplo

Implementar:

- Rota `/test/create-sample-products`
- 20-30 produtos de exemplo por loja

- Diferentes categorias (eletrônicos, roupas, casa, etc.)
- Produtos com e sem código de barras

🎯 PRÓXIMOS PASSOS - PRIORIDADE 2

3. API de Inventário (45-60 min)

Objetivo: Sistema completo de inventário físico

Implementar:

- `GET /api/v1/inventory/lists` - Listar inventários
- `POST /api/v1/inventory/lists` - Criar inventário
- `GET /api/v1/inventory/lists/{id}` - Obter inventário específico
- `POST /api/v1/inventory/lists/{id}/items` - Adicionar produtos ao inventário
- `GET /api/v1/inventory/lists/{id}/items` - Listar itens do inventário
- `POST /api/v1/inventory/items/{id}/count` - Registrar contagem
- `GET /api/v1/inventory/items/{id}/counts` - Histórico de contagens

4. Frontend de Login (30 min)

Objetivo: Interface web para login

Implementar:

- `frontend/login.html` - Tela de login
- `frontend/assets/js/auth.js` - Lógica de autenticação
- `frontend/assets/css/login.css` - Estilos
- Integração com API de login
- Armazenamento seguro do JWT token
- Redirecionamento após login

📱 PRÓXIMOS PASSOS - PRIORIDADE 3

5. Dashboard Web (45 min)

Objetivo: Interface principal do sistema

Implementar:

- `frontend/dashboard.html` - Dashboard principal
- Listagem de produtos

- Criar/editar produtos
- Iniciar inventários
- Estatísticas básicas

6. Scanner de Código de Barras (30 min)

Objetivo: Leitura real de códigos de barras

Implementar:

- Integração com QuaggaJS ou ZXing-js
- Interface de scanner
- Busca automática de produtos por código

🔧 COMANDOS ESSENCIAIS

Iniciar Sistema

```bash
cd \capul_inventario\
docker-compose up -d
```

Ver Status

```bash
docker-compose ps
docker-compose logs backend
```

Endpoints de Teste

```bash
```

```
# Health check
http://localhost:8000/health

# Documentação
http://localhost:8000/docs

# Login (POST)
http://localhost:8000/api/v1/auth/login
Body: {"username": "admin", "password": "admin123"}

# Teste de banco
http://localhost:8000/test/database
```

Reiniciar Backend

```bash
docker-compose restart backend
```

📝 INFORMAÇÕES IMPORTANTES

Credenciais de Teste

- **Login:** admin
- **Senha:** admin123
- **Role:** ADMIN
- **Store:** Loja Matriz (código 001)

Configurações de Banco

- **Host:** localhost:5432
- **Database:** inventario_protheus
- **User:** inventario_user
- **Password:** inventario_2024!
- **Schema:** inventario

Estrutura de Tabelas

```sql
```

```
stores        - Lojas/filiais
users         - Usuários do sistema
products      - Produtos por loja
inventory_lists - Listas de inventário
inventory_items - Itens para contagem
countings     - Registros de contagem
discrepancies  - Divergências encontradas
```

🎯 SUGESTÃO PARA NOVO CHAT

Formato da Mensagem de Abertura:

```
Olá! Estou continuando o desenvolvimento do Sistema de Inventário Protheus.

STATUS ATUAL:
✅ Docker Compose funcionando (PostgreSQL + FastAPI)
✅ Sistema de autenticação JWT implementado
✅ Login admin/admin123 funcionando
✅ Modelos SQLAlchemy criados
✅ Documentação automática em /docs

PRÓXIMO OBJETIVO:
Implementar API de Produtos com CRUD completo e autenticação JWT.

ARQUIVOS PRINCIPAIS CRIADOS:
- docker-compose.yml, .env
- backend/app/main.py (FastAPI principal)
- backend/app/core/database.py (conexão PostgreSQL)
- backend/app/core/security.py (sistema JWT)
- backend/app/models/models.py (modelos SQLAlchemy)
- backend/app/api/auth.py (rotas de autenticação)

O sistema está rodando em:
- Backend: http://localhost:8000
- Docs: http://localhost:8000/docs
- PostgreSQL: localhost:5432

Preciso implementar a API de produtos. Pode me ajudar?
```

💡 DICAS IMPORTANTES

1. **Sempre verificar se Docker está rodando** antes de começar
2. **Usar /docs para testar APIs** - muito mais fácil que curl

3. **Logs são seus amigos**: `docker-compose logs backend`
4. **Banco já tem dados de teste** - 1 loja e 1 usuário admin
5. **JWT token expira em 1h** - renovar se necessário
6. **Sistema já está production-ready** - só falta implementar funcionalidades

---

🚀 EXPECTATIVA DE TEMPO

- **API Produtos:** 30-45 min
- **Frontend Login:** 30 min
- **API Inventário:** 45-60 min
- **Dashboard Web:** 45 min
- **Scanner Códigos:** 30 min

Total estimado: 3-4 horas para sistema completo e funcional!

⚡ RESULTADO FINAL ESPERADO

Um sistema web completo de inventário físico com:

- Login seguro JWT
- Gestão de produtos
- Criação de listas de inventário
- Contagem via web/mobile
- Controle por lojas
- Scanner de códigos de barras
- Relatórios e estatísticas
- Integração com Protheus (futura)

Sistema profissional, escalável e production-ready! 🎉